Database Normalization

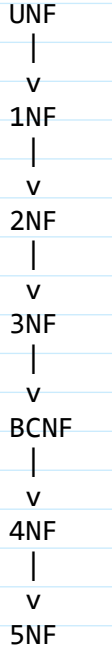
06 October 2026 18:57

Normalization is a systematic technique used in relational database design to organize data into well-structured tables.

The main goal are to:

- Reduce data redundancy.
- Prevent insertion anomalies.
- Prevent update anomalies.
- Prevent deletion anomalies.
- Improve data integrity.
- To make dependencies between Data clear.
- Produce a table that are easier to maintain.

The normal forms are usually studied in this order:



Each normal form imposes additional condition on the database design.

Why Normalization is Required?

Consider this table:

Student_id	Name	Course_id	Course_name	Instructor
101	Alice Johnson	C101	Database Management	Dr. Smith
101	Alice Johnson	C102	Data Structures	Prof. Brown
101	Alice Johnson	C103	Computer Networks	Dr. Wilson
102	Bob Williams	C101	Database Management	Dr. Smith
102	Bob Williams	C104	Operating Systems	Prof. Taylor
103	Charlie Davis	C105	Software Engineering	Dr. Anderson
103	Charlie Davis	C106	Web Development	Prof. Thomas
104	Diana Miller	C102	Data Structures	Prof. Brown
105	Ethan Clark	C107	Machine Learning	Dr. Garcia
105	Ethan Clark	C108	Cloud Computing	Prof. Martin

There is redundancy.
For example:
Student_id = 101
Name = Alice Johnson
Is repeated.
Similarly:
C101 -> Database Management->Dr.Smith
is repeated for every student taking C101.

C102 ->Data Structures -> Prof.Brown
is repeated for every student taking C102.

This can produce several problems.

Objectives of Normalization

- **Reduce Data redundancy:** Avoid storing the same information repeatedly.
- **Prevent insertion anomalies:** Allow new information to be inserted without requiring unrelated information.
- **Prevent update anomalies:** Ensure a change does not need to be made in multiple places.
- **Prevent deletion anomalies:** Avoid accidentally losing useful information when deleting a record.
- **Improve data integrity:** Keep relationship and facts consistent.

Understanding database dependencies

1. **Functional dependency(FD):** Of functional dependency exists when the value of one attribute uniquely determines the value of another attribute.
   It is represented as:

   X -> Y

   Here X is determinant and Y is the dependent attribute.

   For example, consider a Student table:

StudentID	StudentName	Department
101	Amit	CSE

102	Neha	Commerce
103	Ravi	CSE

Assuming each StudentID identifies exactly one student.

StudentID -> StudentName

StudentID -> Department

This means that knowing the StudentID determines the students name and department.
However, the reverse is not necessarily true.

StudentName -> StudentID

Because two students may have the same name.

Types of dependencies

1. **Functional Dependency:** One attribute or group of attributes determines another attribute.
   Example: StudentID -> StudentName

2. **Full functional dependency:** An attribute depends on the entire composite key, not just a part of it. Example: (StudentID, CourseID) -> Grade, where neither StudentID alone nor CourseID alone determines the grade for a particular student-course enrollment.

3. **Partial dependency:** A non-key attribute depends on only part of a composite candidate key.
   Example: (StudentID, CourseID) -> StudentName but actually StudentID -> StudentName.

Consider Employee_Project table

Employee_ID	Project_ID	Employee_Name	Project_Name	Hours_Worked
E101	P01	Alice	AI Alpha	20
E101	P02	Alice	Cloud Beta	15
E102	P01	Harry	AI Alpha	30

Composite Primary Key: {Employee_ID, Project_ID)
Non-Prime Attributes: Employee_Name, Project_Name, Hours_Worked

Identifying the Partial Dependencies

(EmployeeID, Project_ID) → Employee_Name
(EmployeeID, Project_ID) → Project_Name

1) **Employee_ID -> Employee_Name**
   Issue: You only need Employee_ID to know employee's name. You don't need Project_ID. **This is partial dependency.**

2) **Project_ID -> Project_Name**
   Issue: You only need Project_Id to know project name. You don't need Employee_ID. **This is partial dependency.**

3) **{Employee_ID, Project_ID} -> Hours_worked**
   Valid: Ours worked depends on both who the employee is and which project they are working on. This is a **full functional dependency.**

4. **Transitive dependency:** An attribute depends on a key indirectly through another non - key attribute. Example: StudentID -> DeptID and DeptID->DeptName; therefore, StudentID -> DeptName

5. **Multivalued dependency(MVD):** One attribute determines a set of values for another attribute, independently of other attributes. It is represented as X ->> Y. Example: A student may have multiple independent hobbies and multiple independent languages.

   Another example of Multi-valued Dependency

   Consider a table Student

StudentID(X)	Course(Y)	Hobby(Z)
101	Computer Science	Music
101	Computer Science	Sports
101	Mathematics	Music
101	Mathematics	Sports

   Why is this a Multivalued Dependency?
   For student _ID. 101 There is a set of courses. {Computer Science, Mathematics } And a set of hobbies. {music, sports.} Because these two sets are completely independent, the database is forced to repeat every combination of course and hobby for that student.

   If student 101 decides to pick up a new hobby like painting, we cannot just add one row. We would have to insert two new rows to pair painting with both computer science and mathematics.

   This gives us two MVDs:
   i. Student_ID ->> Course
   ii. Student_ID ->> Hobby

Keys used in normalization

- **SuperKey**: Any attribute or combination of attributes that uniquely identifies a row.
- **Candidate Key:** A minimal super key. Removing any attribute makes it no longer a super key.
- **Primary Key:** The candidate key selected to identify rows.
- **Prime Attribute:** An attribute that belongs to at least one candidate key.
- **Non-Prime Attribute:** An attribute that does not belongs to any candidate key.